

Progetto di Reti di Calcolatori

Focusly

SPA con architettura Client-Server per la pianificazione delle attività

Samuele Caporale
Matricola 329096
A.A. 2025/2026

Indice

1	Introduzione	3
2	Specifica di Progetto	4
2.1	Requisiti funzionali	4
2.2	Requisiti non funzionali	4
3	Architettura e tecnologie utilizzate	5
3.1	Tecnologie utilizzate	5
3.2	Workspace	6
3.3	Architettura e Type of Service	7
4	Analisi ed Implementazione	8
4.1	Scelte implementative	8
4.2	Autenticazione e gestione della sessione	8
4.3	Persistenza cloud, modellazione dati e sicurezza	8
4.4	Organizzazione del frontend e business logic	9
4.5	Flussi applicativi principali	10
4.6	Difficoltà, limiti e compromessi progettuali	11
5	Conclusioni	12
6	Documentation	13

1 Introduzione

Focusly nasce come progetto orientato alla pianificazione dello studio attraverso una web application in grado di organizzare task, gruppi tematici e milestone all'interno di un'unica interfaccia. L'idea di partenza è offrire uno strumento semplice per l'utente finale, ma progettato secondo criteri moderni, con autenticazione remota, persistenza cloud e possibilità di mantenere una continuità d'uso anche in presenza di dati locali salvati nel browser.

La scelta di sviluppare e analizzare questo progetto nel contesto di Reti di Calcolatori deriva dall'interesse per le applicazioni web che affidano parte rilevante della loro logica a servizi distribuiti raggiunti tramite rete. In Focusly, infatti, la componente significativa non riguarda soltanto l'interfaccia grafica, ma soprattutto il modo in cui il browser interagisce con servizi cloud esterni per autenticare l'utente, memorizzare il workspace personale e riallineare le informazioni tra sessioni diverse.

La presente relazione ha quindi l'obiettivo di descrivere la specifica del progetto, le tecnologie adottate e le principali scelte architetturali, mettendo in evidenza i motivi per cui è stata preferita una soluzione client-cloud basata su Firebase.

2 Specifica di Progetto

L'obiettivo del progetto é la realizzazione di un'applicazione web per la pianificazione e gestione di task organizzati in gruppi tematici, con milestone e statistiche di analisi. La specifica si concentra quindi su un sistema che unisca semplicità d'uso, accesso remoto ai dati e una struttura abbastanza modulare da poter evolvere nel tempo.

2.1 Requisiti funzionali

Dal punto di vista funzionale, l'applicazione deve consentire agli utenti di:

- Autenticarsi tramite account Google.
- Creare, modificare e cancellare task, gruppi e milestone.
- Visualizzare statistiche aggregate sui task completati e sulle milestone raggiunte.
- Salvare e sincronizzare i dati su più dispositivi tramite cloud.
- Consentire l'utilizzo offline con sincronizzazione successiva.

2.2 Requisiti non funzionali

Il sistema deve rispettare alcuni requisiti non funzionali:

- Essere responsive e compatibile con i principali browser.
- Essere sicuro, isolando i dati degli utenti tramite regole di accesso.
- Essere efficiente, minimizzando le operazioni di rete e ottimizzando le prestazioni.

3 Architettura e tecnologie utilizzate

3.1 Tecnologie utilizzate

Il progetto combina tecnologie web di base, librerie frontend, protocolli di trasporto e servizi cloud. L'insieme risulta coerente con una Single Page Application, nella quale gran parte della logica applicativa viene eseguita sul client, mentre autenticazione e dati persistenti vengono delegati a servizi remoti specializzati.

- **HTML5, CSS3 e JavaScript ES Modules**
Costituiscono la base della SPA eseguita nel browser, definendo struttura, stile e logica applicativa.
- **React 18 e ReactDOM**
Realizzano il rendering dell'interfaccia e la composizione dei componenti della dashboard.
- **Vite e @vitejs/plugin-react**
Gestiscono l'ambiente di sviluppo, il bundling e la compilazione del frontend React.
- **Bootstrap 5**
Fornisce griglia, utility CSS e componenti visuali per il layout responsive.
- **Bootstrap Icons**
Mette a disposizione le icone svg usate nell'interfaccia utente.
- **Zustand**
Implementa lo store globale che coordina stato, bootstrap e azioni asincrone.
- **Chart.js**
Genera i grafici statistici mostrati nella sezione analytics dell'app.
- **HTTPS**
È il protocollo di trasporto usato per le comunicazioni sicure tra browser e servizi remoti.
- **Firebase JavaScript SDK**
Libreria client che integra nel frontend autenticazione e accesso ai servizi cloud.
- **Firebase Authentication**
Gestisce il login con Google e il mantenimento dello stato di sessione utente.
- **Cloud Firestore**
Database NoSQL cloud usato per salvare profili, task, gruppi e milestone.
- **localStorage**
Offre persistenza locale nel browser e supporta la migrazione iniziale dei dati verso Firestore.
- **Codex**
Strumento di supporto alla generazione del codice e all'esecuzione di operazioni sul file system. È stato utilizzato attraverso agenti specializzati, configurati per attività di progettazione UI/UX, sviluppo frontend e coordinamento del flusso di lavoro, con l'obiettivo di accelerare alcune fasi del processo di sviluppo.

3.2 Workspace

L'architettura prevede un workspace organizzato in più aree: il frontend React costituisce il cuore dell'applicazione, Firebase fornisce i servizi remoti di autenticazione e persistenza, mentre la documentazione e gli script di supporto completano l'organizzazione del repository. La struttura attuale del progetto è la seguente:

```
root
├── docs/
│   └── notes/
├── frontend/
│   ├── public/
│   ├── node_modules/
│   ├── src/
│   │   ├── components/
│   │   ├── hooks/
│   │   ├── lib/
│   │   ├── model/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── state/
│   │   └── utils/
│   ├── firebase.json
│   ├── firestore.rules
│   └── vite.config.js
├── scripts/
└── package.json
```

La directory **docs/** raccoglie la documentazione e le specifiche architetturali per agenti codex. La directory **frontend/** ospita l'intera app, insieme alla configurazione del client React e dei servizi Firebase, mentre **scripts/** contiene script di supporto al workspace, utili per lo sviluppo locale e per operazioni accessorie come l'esportazione di dati. I file **firebase.json**, **firestore.rules** e **vite.config.js** sono file di configurazione rispettivamente per l'ambiente Firebase, le regole di sicurezza Firestore e il bundler Vite, mentre **package.json** definisce metadati, dipendenze e script di esecuzione del progetto frontend.

All'interno del frontend, la directory **public/** contiene le risorse esposte pubblicamente al client come immagini raster e icone vettoriali, la directory **node_modules/** raccoglie le dipendenze installate del progetto e i pacchetti necessari al funzionamento dell'ambiente React/Vite, mentre la directory **src/** contiene il codice sorgente dell'applicazione React. In particolare, **components/** contiene i componenti visuali riutilizzabili dell'interfaccia, organizzati per area funzionale; **hooks/** definisce gli hook React usati per incapsulare logiche riutilizzabili, come la gestione dei task, dei gruppi, del timer o di comportamenti della UI; **lib/** contiene la configurazione di Firebase e l'inizializzazione delle librerie condivise, in modo da centralizzare la configurazione; **model/** contiene i modelli dati dell'applicazione, come task e gruppi, mantenendo una struttura coerente del dominio; **pages/** contiene i contenitori ad alto livello delle schermate dell'intera SPA; **services/** contiene i servizi che gestiscono le operazioni di business logic, autenticazione, persistenza Firestore, migrazione e analisi delle statistiche; **state/** comprende lo store globale Zustand e la logica di coordinamento dello stato condiviso dell'applicazione; **utils/** racchiude funzioni di utilità come helper per formattazione, gestione delle date e temi.

Il frontend costituisce il nodo applicativo principale. Il flusso operativo di autenticazione, sincronizzazione e persistenza dei dati avviene direttamente tra browser e servizi Firebase.

3.3 Architettura e Type of Service

Dal punto di vista delle reti di calcolatori, Focusly implementa un'architettura **Client-Server**, nello specifico Client-Cloud, dove il browser esegue la Single Page Application React e comunica tramite HTTPS con i servizi remoti messi a disposizione da Firebase.

Questo, dal punto di vista dello sviluppatore, rientra nella tipologia di servizio **BaaS** in quanto il backend non viene implementato direttamente all'interno del progetto, ma viene delegato alla piattaforma Firebase, che fornisce tramite SDK servizi già pronti di autenticazione e persistenza dei dati.

Le interazioni di rete presenti nel sistema sono principalmente di due tipi:

- autenticazione remota con Google tramite popup, mediata da Firebase Authentication;
- accesso al database Firestore mediante letture, scritture e batch write effettuati dall'SDK Firebase.

Questa scelta riduce la complessità infrastrutturale e rende il prototipo di più rapida implementazione, ma comporta anche una maggiore dipendenza dal provider cloud adottato.

Uno degli aspetti principali a favore della scelta architetture sta nella sincronizzazione dei dati: Viene delegata prevalentemente al server di Firebase.

4 Analisi ed Implementazione

4.1 Scelte implementative

Il progetto è stato inizialmente sviluppato come prototipo minimo, con l'obiettivo di validare rapidamente le funzionalità principali dell'applicazione. Successivamente, il sistema è stato esteso in maniera incrementale, mantenendo il codice versionato tramite Git e verificando progressivamente il corretto funzionamento attraverso test end-to-end.

L'adozione di un modello *serverless* client-driven, nel quale il frontend interagisce direttamente con Firebase, ha consentito una rapida implementazione delle funzionalità di base e ha reso possibile utilizzare l'applicazione già durante le fasi di sviluppo.

4.2 Autenticazione e gestione della sessione

La configurazione Firebase è centralizzata in `frontend/src/lib/firebase.js`. Il modulo legge le variabili d'ambiente `VITE_FIREBASE_*`, inizializza l'app Firebase e crea tre oggetti condivisi: `auth`, `db` e `googleProvider`. Questa centralizzazione evita duplicazioni, riduce l'accoppiamento e rende più controllabile l'intero ciclo di inizializzazione.

Il provider Google viene inoltre configurato con lingua italiana e con il parametro `prompt = select_account`, in modo da rendere esplicita la scelta dell'account. Il modulo `authService.js` espone quindi le primitive principali per il login, il logout e l'osservazione dello stato di autenticazione tramite `onAuthStateChanged()`.

L'applicazione non effettua interrogazioni continue verso il server per determinare se l'utente sia autenticato: si affida invece al listener di Firebase Auth, che notifica al frontend ogni cambiamento di sessione. Questa soluzione è interessante sia dal punto di vista tecnico sia da quello didattico, perché mostra come sia possibile delegare la gestione dell'identità a un provider affidabile senza dover sviluppare internamente l'intero ciclo di autenticazione.

Nel componente radice `App.jsx`, una `useEffect` registra la subscription all'autenticazione. Quando Firebase segnala un utente autenticato, lo store Zustand richiama l'azione `handleAuthStateChange`, aggiorna o crea il profilo su Firestore, valuta l'eventuale migrazione dei dati locali e infine carica lo stato dell'area di lavoro. In caso di assenza di sessione, l'app mostra invece la schermata di login. La business logic di ingresso e uscita dall'app non è quindi distribuita nei componenti di UI, ma concentrata nello store globale.

4.3 Persistenza cloud, modellazione dati e sicurezza

La persistenza remota è affidata a Cloud Firestore. Il modello dei dati è gerarchico e organizzato per utente: ogni account dispone di uno spazio logico separato, identificato dal proprio `uid`.

```
users/{uid}
users/{uid}/tasks/{taskId}
users/{uid}/groups/{groupId}
users/{uid}/groups/{groupId}/milestones/{milestoneId}
```

La collezione `users` contiene il profilo e i metadati di sincronizzazione, mentre le sottocollezioni `tasks` e `groups` rappresentano il workspace dell'utente; ciascun gruppo include a sua volta le relative milestone. Questa struttura è semplice, ma molto efficace rispetto al problema affrontato, perché garantisce isolamento logico dei dati e si integra bene con le regole di sicurezza basate su `uid`.

Il modulo `firestoreMappers.js` svolge un ruolo fondamentale nella conversione tra oggetti del dominio applicativo e documenti Firestore. Esso si occupa della trasformazione delle date in `Timestamp`, della conversione inversa verso formati leggibili dal frontend e della normalizzazione dei campi principali. Tale scelta evita che i componenti React debbano conoscere il formato interno del database e rappresenta una buona pratica di separazione tra dominio e persistenza.

Il modulo più rilevante per la comunicazione dati è `firestoreService.js`, che costruisce riferimenti tramite `doc()` e `collection()` ed esegue letture, query, aggiornamenti mirati e operazioni atomiche mediante `writeBatch()`. L'uso dei batch write per create, update e delete consente di mantenere coerenti i documenti applicativi e il campo `workspaceRevision` del profilo utente. Ogni modifica incrementa infatti la revisione del workspace, permettendo all'app di distinguere se il *local storage* sia allineato con lo stato remoto.

Anche il tema della sicurezza è stato affrontato in modo coerente. Le regole definite in `frontend/firestore.rules` consentono lettura e scrittura solo in presenza di un utente autenticato, limitano l'accesso ai soli documenti associati al corretto `uid`, impediscono la cancellazione del profilo utente e negano di default ogni altro percorso (rispettando il principio del minimo privilegio). Dal punto di vista delle reti, la scelta più rilevante consiste nello spostare il controllo di accesso dal frontend alle regole server-side di Firestore: anche se il client può essere manipolato, il database continua a imporre isolamento tra utenti.

4.4 Organizzazione del frontend e business logic

Lo store definito in `frontend/src/state/store.js` è il vero orchestratore dell'applicazione. Ospita lo stato condiviso, come `tasks`, `groups`, `selectedGroupId`, `authStatus`, `dataStatus` e `workspaceRevision`, ma anche le principali azioni asincrone che coordinano bootstrap, migrazione, caricamento remoto e aggiornamento dell'interfaccia.

La logica di bootstrap è collocata nello store per un motivo preciso: essa deve coordinare autenticazione, caricamento dei dati, migrazione da `localStorage` e stato della UI. Se fosse dispersa in molti componenti React, aumenterebbe il rischio di inconsistenze.

Il frontend utilizza inoltre un layer di servizi separato: `taskService` applica la logica CRUD dei task, `groupService` gestisce gruppi e stato derivato, `migrationService` governa import e reimport dei dati locali, `analyticsService` calcola statistiche derivate e `storageService` realizza la persistenza locale. La logica risulta quindi stratificata: i componenti React invocano azioni dello store, lo store richiama i servizi di dominio, e solo `firestoreService` conosce i dettagli della persistenza remota.

I modelli `Task.js` e `Group.js` contribuiscono inoltre a normalizzare la forma dei dati. Alcune regole di business particolarmente rilevanti sono la presenza obbligatoria di identificatore, ordine e data pianificata per i task, il vincolo sui livelli di priorità e il fatto che lo stato del gruppo non venga salvato come verità assoluta, ma ricalcolato dinamicamente dai task del giorno. Questa soluzione riduce la ridondanza, ma richiede una progettazione più attenta della logica di aggiornamento.

4.5 Flussi applicativi principali

Il primo login costituisce il flusso di rete più ricco dell'applicazione. La sequenza implementata prevede l'avvio del login Google, l'autenticazione tramite Firebase Auth, la creazione o l'aggiornamento del profilo utente in `users/{uid}`, la verifica dell'eventuale backup locale importabile e il successivo caricamento del workspace. Se la migrazione non è ancora stata eseguita, task, gruppi e milestone vengono copiati in Firestore tramite batch write e il profilo utente viene marcato come già importato.

La migrazione avviene in modalità *merge only*: i documenti già presenti vengono aggiornati per identificatore, mentre quelli remoti assenti nel backup locale non vengono cancellati. Si tratta di una scelta prudente, pensata per ridurre il rischio di perdita dati, ma anche di un esempio concreto di compromesso progettuale: la coerenza assoluta viene sacrificata in favore della sicurezza del contenuto già presente nel cloud.

Quando l'utente crea o modifica un task, il flusso applicativo coinvolge il componente di dashboard, lo store, `taskService`, il servizio Firestore e infine l'aggiornamento del backup locale tramite `syncLocalBackup()`. Un meccanismo analogo viene applicato per update e delete. Il vantaggio è che l'app mantiene coerenti tre livelli distinti: interfaccia React, local storage del browser e stato persistente su Firestore.

La gestione dei gruppi risulta leggermente più complessa per la presenza delle milestone. La creazione salva sia il documento del gruppo sia la sottocollezione `milestones`; l'aggiornamento usa `syncMilestonesForGroup()` per creare, modificare o rimuovere milestone obsolete; la cancellazione del gruppo implica anche la rimozione dei task collegati e delle milestone associate. Questo flusso coinvolge contemporaneamente struttura dati, persistenza remota e aggiornamento dello stato locale.

L'app espone infine una funzione di reimport manuale dalla schermata impostazioni. Tale flusso è utile per riallineare vecchi dati locali con Firestore, ma include una protezione contro account diversi: se il backup locale risulta associato a un altro `uid`, il reimport viene bloccato esplicitamente. Anche questa scelta evidenzia attenzione alla sicurezza e alla correttezza del dato.

4.6 Difficoltà, limiti e compromessi progettuali

Durante lo sviluppo sono emersi tre punti critici concreti.

Il primo è stato mantenere coerenti stato React, `localStorage` e Firestore: per evitare conflitti è stato introdotto il salvataggio centralizzato con `syncLocalBackup()`, il controllo dell'`uid` del backup locale e una migrazione `mergeOnly` in `importLocalData()`, così da aggiornare i documenti esistenti senza cancellare quelli remoti.

Il secondo ha riguardato le operazioni composte sui gruppi: `updateGroup()` deve sincronizzare anche la sottocollezione `milestones` tramite `syncMilestonesForGroup()`, mentre `deleteGroup()` va coordinata con `deleteTasksByGroup()` per rimuovere nello stesso flusso milestone e task collegati, usando `writeBatch()` e gli ID già presenti nello store per evitare query aggiuntive.

Il terzo ha riguardato i costi di lettura: nella prima versione il bootstrap poteva arrivare a circa 5.000–10.000 letture Firestore, poi ridotte a circa 500 in uso normale mantenendo `localStorage` come cache persistente e riusandola solo quando `uid` e `workspaceRevision` coincidono con il profilo remoto.

La soluzione adottata presenta comunque diversi punti di forza:

- autenticazione delegata a un provider affidabile;
- isolamento dei dati tramite regole Firestore per utente;
- uso di batch write per operazioni atomiche;
- *local storage* utile per velocizzare il bootstrap;
- separazione netta tra UI, store, servizi di dominio e persistenza.

Esistono tuttavia anche alcuni limiti strutturali:

- il caricamento dati usa letture esplicite, non listener realtime Firestore;
- l'assenza di un backend custom rende più difficile centralizzare la business logic.
- alcune operazioni complesse, come la cancellazione coordinata di gruppo e task, rimangono gestite interamente nel client;
- la sincronizzazione multi-dispositivo si basa sul refresh e sulla persistenza in cloud dei dati e non su push continuo verso la UI dal server.

Focusly rappresenta quindi un esempio di architettura *serverless client-driven*: semplice, efficace per un progetto "in the small", ma meno controllabile di una soluzione con backend applicativo dedicato. Il compromesso è stato scelto consapevole: privilegiare rapidità di sviluppo, chiarezza architetturale e integrazione con servizi gestiti, accettando i limiti di una minore centralizzazione della logica lato server.

5 Conclusioni

Gli obiettivi iniziali del progetto possono dirsi raggiunti in misura significativa. Focusly consente infatti di autenticarsi con account Google, creare e gestire task, gruppi e milestone, visualizzare statistiche e mantenere la persistenza dei dati nel cloud, offrendo al tempo stesso un supporto locale utile per il bootstrap e la migrazione. La congruenza tra specifica iniziale e risultato ottenuto appare quindi complessivamente soddisfacente.

Dal punto di vista grafico, l'applicazione risulta coerente ai requisiti non funzionali, in quanto:

- Risulta Responsive in versione Desktop e Mobile, necessita solo di qualche rifinitura su i breakpoint meno comuni e ciò è accettabile in quanto prototipo.
- Risulta Sicuro dal punto di vista delle operazioni sui dati, in quanto ogni utente può accedere con il minimo privilegio ai soli dati reattivi al proprio *uid*.

I possibili sviluppi futuri più sensati, riguardano: meccanismi realtime per la sincronizzazione continua e collaborazione tra più utenti, le funzionalità di analisi e il calendario per visualizzare la pianificazione completa su scala più ampia.

In questa prospettiva, Focusly costituisce un prototipo: non un sistema definitivo, ma un progetto coerente e ben impostato, capace di mostrare come le tecnologie cloud moderne possano essere impiegate in modo efficace all'interno di un'applicazione web orientata all'analisi e miglioramento della produttività personale.

6 Documentation

- React Documentation, <https://react.dev/>
- Vite Documentation, <https://vite.dev/guide/>
- Firebase Documentation, <https://firebase.google.com/docs>
- Firebase Authentication Documentation, <https://firebase.google.com/docs/auth>
- Cloud Firestore Documentation, <https://firebase.google.com/docs/firestore>
- Zustand Documentation, <https://zustand.docs.pmnd.rs/>
- Bootstrap Documentation, <https://getbootstrap.com/docs/5.3/>
- Chart.js Documentation, <https://www.chartjs.org/docs/latest/>